

INHERITANCE

Aim:

To understand and implement inheritance concepts in Java.

PRE LAB EXERCISE

QUESTIONS

- ✓ What is inheritance?

Inheritance is a feature in Java where one class (child class) acquires the properties and methods of another class (parent class).

It helps in reusing existing code and creating a relationship between classes.

Example:

```
class Child extends Parent {}
```

- ✓ What is code reusability?

Code reusability means using already written code again without rewriting it.

In inheritance, a child class can reuse the fields and methods of the parent class, which saves time and reduces duplication.

- ✓ What is the use of extends keyword?

The extends keyword is used to inherit the properties and methods of a parent class into a child class.

Syntax:

```
class ChildClass extends ParentClass {}
```

It creates an "is-a" relationship between the classes.

Example:

```
class Result extends Student {}
```

Here, Result inherits from Student.

IN LAB EXERCISE

Objective:

To implement all types of inheritance.

PROGRAMS:

Student Result System (Single Inheritance)

Question:

A school wants to store student details and calculate marks. Create a base class Student and a derived class Result.

Code:

```
class Student {  
    String name;  
    int rollNo;  
  
    void getDetails() {  
        name = "Anitha";  
        rollNo = 101;  
    }  
}  
  
class Result extends Student {  
    int marks = 85;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Marks: " + marks);  
    }  
}
```

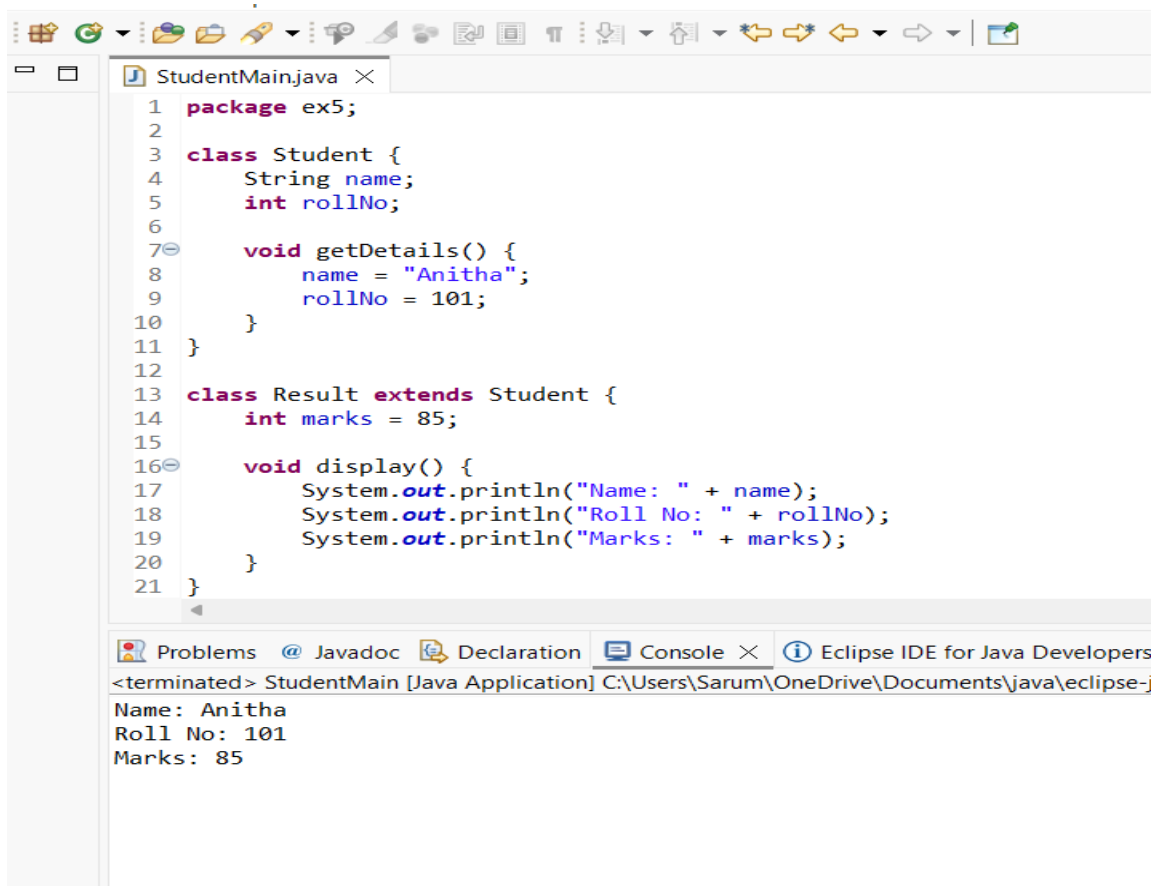
```
public class Main {  
    public static void main(String[] args) {  
        Result r = new Result();  
        r.getDetails();  
        r.display();  
    }  
}
```

Output:

Name: RAM

Roll No: 101

Marks: 85



The screenshot shows the Eclipse IDE interface. The top toolbar contains various icons for file operations, editing, and running. The main editor window displays the file 'StudentMain.java' with the following code:

```
1 package ex5;  
2  
3 class Student {  
4     String name;  
5     int rollNo;  
6  
7     void getDetails() {  
8         name = "Anitha";  
9         rollNo = 101;  
10    }  
11 }  
12  
13 class Result extends Student {  
14     int marks = 85;  
15  
16     void display() {  
17         System.out.println("Name: " + name);  
18         System.out.println("Roll No: " + rollNo);  
19         System.out.println("Marks: " + marks);  
20     }  
21 }
```

Below the editor, the 'Console' tab is active, showing the output of the program:

```
<terminated> StudentMain [Java Application] C:\Users\Sarum\OneDrive\Documents\java\eclipse-  
Name: Anitha  
Roll No: 101  
Marks: 85
```

2. Bank Account System (Hierarchical Inheritance)

Question:

A bank has Savings and Current accounts. Both inherit from a common Account class.

Code:

```
class Account {  
    void showAccountType() {  
        System.out.println("Bank Account");  
    }  
}  
  
class SavingsAccount extends Account {  
    void interest() {  
        System.out.println("Savings Account gives interest");  
    }  
}  
  
class CurrentAccount extends Account {  
    void overdraft() {  
        System.out.println("Current Account supports overdraft");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        SavingsAccount s = new SavingsAccount();  
        CurrentAccount c = new CurrentAccount();  
  
        s.showAccountType();  
        s.interest();  
    }  
}
```

```

        c.showAccountType();

        c.overdraft();
    }
}

```

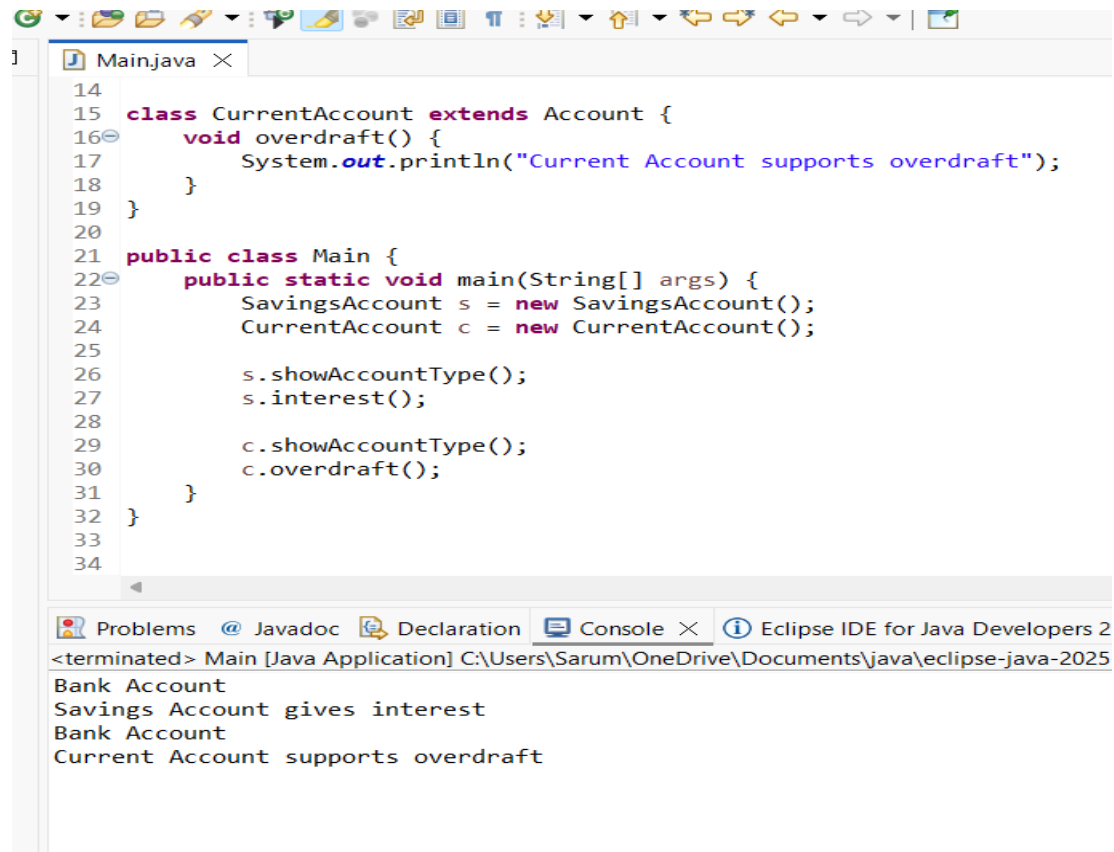
Output:

Bank Account

Savings Account gives interest

Bank Account

Current Account supports overdraft



The screenshot shows the Eclipse IDE with a Java project. The editor displays the following code in `Main.java`:

```

14
15 class CurrentAccount extends Account {
16     void overdraft() {
17         System.out.println("Current Account supports overdraft");
18     }
19 }
20
21 public class Main {
22     public static void main(String[] args) {
23         SavingsAccount s = new SavingsAccount();
24         CurrentAccount c = new CurrentAccount();
25
26         s.showAccountType();
27         s.interest();
28
29         c.showAccountType();
30         c.overdraft();
31     }
32 }
33
34

```

The console output at the bottom shows the execution results:

```

<terminated> Main [Java Application] C:\Users\Sarum\OneDrive\Documents\java\eclipse-java-2025
Bank Account
Savings Account gives interest
Bank Account
Current Account supports overdraft

```

3. Vehicle System (Multilevel Inheritance)

Rithanya K V
24BCS226

Question:

A company classifies vehicles as Vehicle → Car → ElectricCar.

Code:

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void fuelType() {
        System.out.println("Car uses petrol");
    }
}

class ElectricCar extends Car {
    void battery() {
        System.out.println("Electric car uses battery");
    }
}

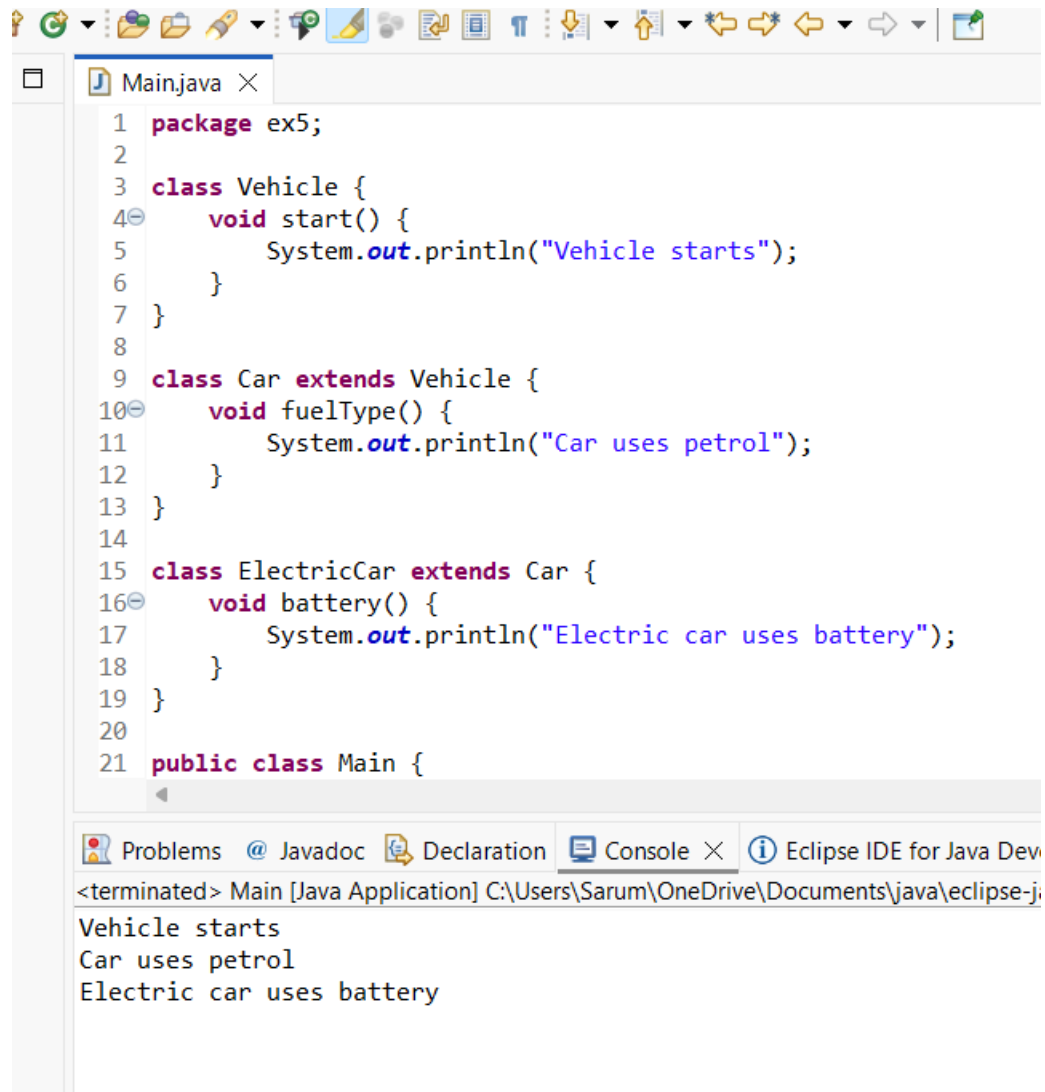
public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar();
        e.start();
        e.fuelType();
        e.battery();
    }
}
```

Output:

Vehicle starts

Car uses petrol

Electric car uses battery



The screenshot shows the Eclipse IDE interface. The top toolbar contains various icons for file operations and development tools. The main editor window displays a file named 'Main.java' with the following code:

```
1 package ex5;
2
3 class Vehicle {
4     void start() {
5         System.out.println("Vehicle starts");
6     }
7 }
8
9 class Car extends Vehicle {
10    void fuelType() {
11        System.out.println("Car uses petrol");
12    }
13 }
14
15 class ElectricCar extends Car {
16    void battery() {
17        System.out.println("Electric car uses battery");
18    }
19 }
20
21 public class Main {
```

Below the editor, the 'Console' tab is active, showing the output of the program:

```
<terminated> Main [Java Application] C:\Users\Sarum\OneDrive\Documents\java\eclipse-j
Vehicle starts
Car uses petrol
Electric car uses battery
```

POST LAB EXERCISE

- ✓ Why Java does not support multiple inheritance using classes and how it is implemented?

Java does not support multiple inheritance using classes to avoid ambiguity problems (Diamond Problem).

If two parent classes have the same method, the child class will not know which method to inherit.

To solve this problem, Java supports multiple inheritance using **interfaces**.

Example:

```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}  
  
class C implements A, B {  
    public void show() {  
        System.out.println("From A");  
    }  
  
    public void display() {  
        System.out.println("From B");  
    }  
}
```

Here, class C inherits from multiple interfaces.

- ✓ What is the role of the super keyword? Give examples.

The super keyword is used to refer to the parent class.

It is mainly used to:

- Access parent class variables

- Call parent class methods

- Call parent class constructor

Example 1 – Access parent variable:

```
class Parent {  
    int x = 10;  
}  
  
class Child extends Parent {  
    void show() {  
        System.out.println(super.x);  
    }  
}
```

- ✓ Can a child class access private members of the parent class? Why?

No, a child class cannot directly access private members of the parent class.

Because private members are accessible only within the same class.

However, they can be accessed indirectly using public getter and setter methods.

- ✓ Explain why hybrid inheritance is not supported in Java.

Hybrid inheritance is a combination of more than one type of inheritance (like multiple + multilevel).

Java does not support hybrid inheritance using classes because it includes multiple inheritance, which may cause ambiguity.

However, hybrid inheritance can be achieved using interfaces.

Result:

Thus the different types of inheritance were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		